

1.processid

```
bindu@Bindusree:~$ nano myprocessid.c
bindu@Bindusree:~$ gcc myprocessid.c -o myprocessid
bindu@Bindusree:~$ ./myprocessid
My Process ID (PID) = 563
bindu@Bindusree:~$
```

```
GNU nano 8.7.1 myprocessid.c
#include <stdio.h>
#include <unistd.h>
int main()
{
printf("My Process ID (PID) = %d\n", getpid());
return 0;
}
```

2.creating child using fork()

```
bindu@Bindusree:~$ nano childusingfork.c
bindu@Bindusree:~$ gcc childusingfork.c -o childusingfork
bindu@Bindusree:~$ ./childusingfork
Parent Process
PID = 685
Child Process
PID = 686
bindu@Bindusree:~$
```

```
bindu@Bindusree: ~  
GNU nano 8.7.1 childusingfork.c  
#include <stdio.h>  
#include <unistd.h>  
int main()  
{  
    pid_t pid = fork();  
    if(pid == 0)  
    {  
        printf("Child Process\n");  
        printf("PID = %d\n", getpid());  
    }  
    else  
    {  
        printf("Parent Process\n");  
        printf("PID = %d\n", getpid());  
    }  
    return 0;  
}
```

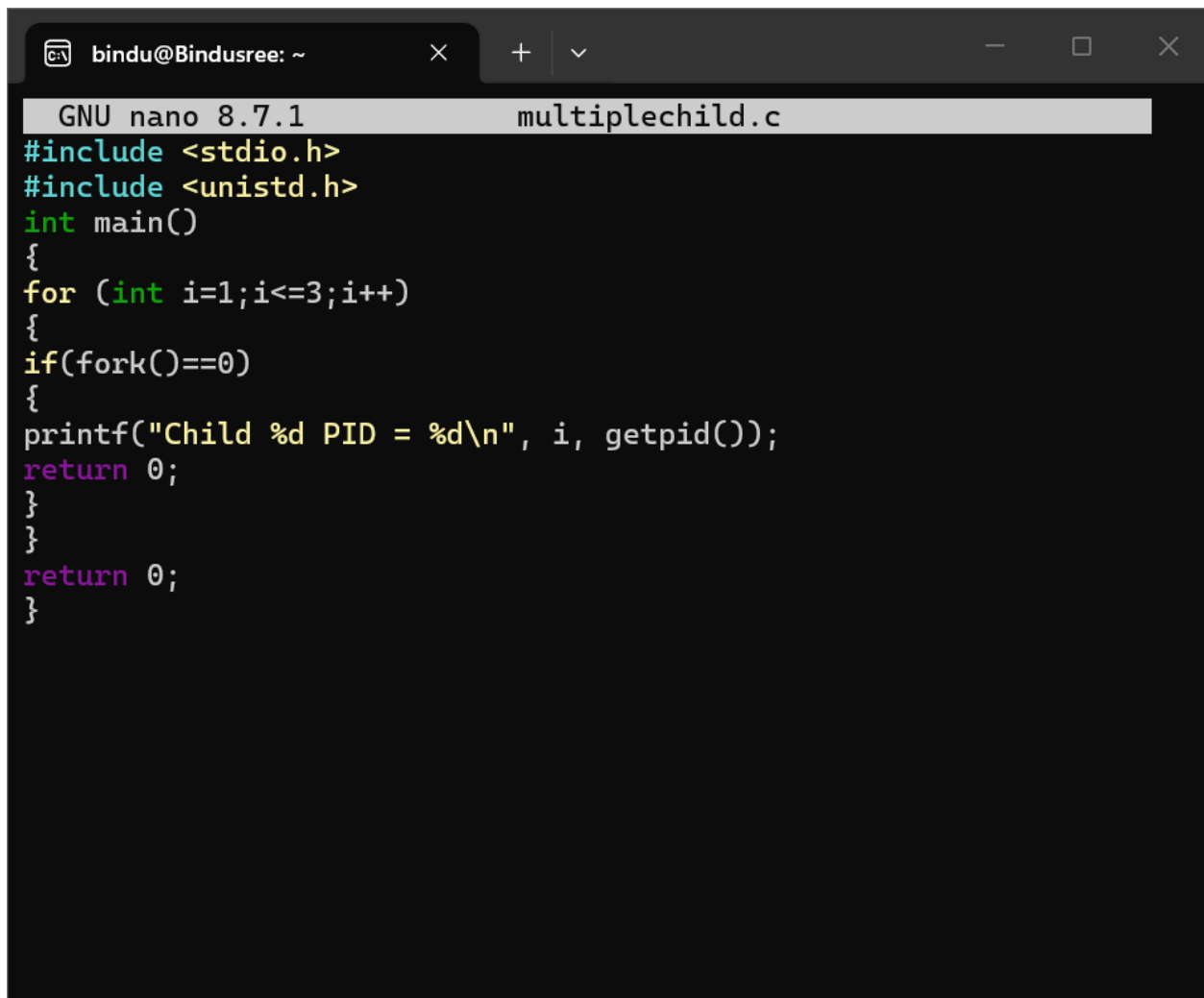
3. Parent Waits for child(wait())

```
bindu@Bindusree:~$ nano parentwaitsforchild.c  
bindu@Bindusree:~$ gcc parentwaitsforchild.c -o parentwaitsforchild  
bindu@Bindusree:~$ ./parentwaitsforchild  
parentwaitsforchild: command not found  
bindu@Bindusree:~$ |
```

```
bindu@Bindusree: ~  
GNU nano 8.7.1 parentwaitsforchild.c  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
int main()  
{  
    pid_t pid = fork();  
    if(pid == 0)  
    {  
        printf("Child Process Executing...\n");  
        sleep(3);  
        printf("Child Finished\n");  
    }  
    else  
    {  
        wait(NULL);  
        printf("Parent Process Continues\n");  
    }  
    return 0;  
}
```

4. Multiple Child Processes

```
bindu@Bindusree:~$ nano multiplechild.c  
bindu@Bindusree:~$ gcc multiplechild.c -o multiplechild  
bindu@Bindusree:~$ ./multiplechild  
.multiplechild: command not found  
bindu@Bindusree:~$
```

A screenshot of a terminal window with a dark background. The window title bar shows 'bindu@Bindusree: ~' and standard window controls. The terminal displays the GNU nano 8.7.1 editor editing a file named 'multiplechild.c'. The code is a C program that uses 'fork()' to create three child processes. It includes headers for 'stdio.h' and 'unistd.h'. The 'main' function contains a loop from i=1 to i=3. Inside the loop, it checks if 'fork()' returns 0 (child process). If so, it prints the child's PID using 'getpid()' and then returns 0. The parent process also returns 0 at the end of the function. The code is color-coded: keywords are green, strings are yellow, and return values are purple.

```
GNU nano 8.7.1 multiplechild.c
#include <stdio.h>
#include <unistd.h>
int main()
{
for (int i=1;i<=3;i++)
{
if(fork()==0)
{
printf("Child %d PID = %d\n", i, getpid());
return 0;
}
}
return 0;
}
```

5.Linux command using exchlp()

```
bindu@Bindusree:~$ nano linux.c
bindu@Bindusree:~$ gcc linux.c -o linux
bindu@Bindusree:~$ ./linux
Before exec()
childusingfork
childusingfork.c
doublefork
doublefork.c
fork
fork.c
linux
linux.c
multiplechild
multiplechild.c
myprocessid
myprocessid.c
myprocessid.c.save
parent
parent.c
parent.save
parentchild
parentchild.c
parentwaitsforchild
parentwaitsforchild.c
processid
processid.c
username
username.c
```

```
bindu@Bindusree: ~  
GNU nano 8.7.1 linux.c  
#include <stdio.h>  
#include <unistd.h>  
int main()  
{  
printf("Before exec()\n");  
execlp("ls", "ls", "-l", NULL);  
printf("This will not execute.\n");  
return 0;  
}
```

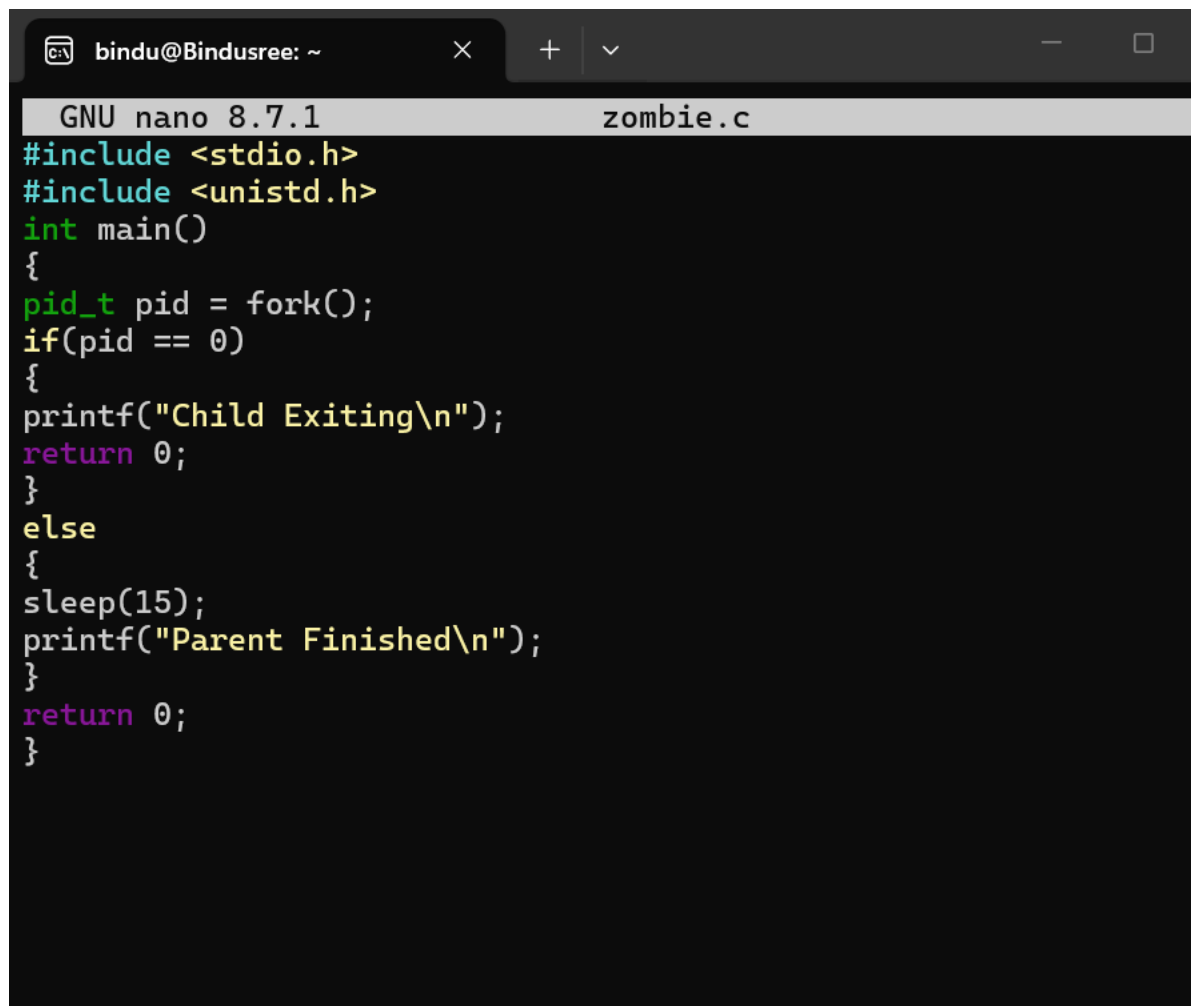
6. Parent and Child executing concurrently

```
bindu@Bindusree: ~$ nano concurrently.c  
bindu@Bindusree: ~$ gcc concurrently.c -o concurrently  
bindu@Bindusree: ~$ ./concurrently  
PID: 1034 Count: 1  
PID: 1035 Count: 1  
PID: 1034 Count: 2  
PID: 1035 Count: 2  
PID: 1035 Count: 3  
PID: 1034 Count: 3  
PID: 1034 Count: 4  
PID: 1035 Count: 4  
PID: 1034 Count: 5  
PID: 1035 Count: 5  
bindu@Bindusree: ~$
```

```
bindu@Bindusree: ~  
GNU nano 8.7.1 concurrently.c  
#include <stdio.h>  
#include <unistd.h>  
int main()  
{  
fork();  
for(int i=1;i<=5;i++)  
{  
printf("PID: %d Count: %d\n", getpid(), i);  
sleep(1);  
}  
return 0;  
}
```

7.zombie process

```
bindu@Bindusree:~$ nano zombie.c  
bindu@Bindusree:~$ gcc zombie.c -o zombie  
bindu@Bindusree:~$ ./zombie  
Child Exiting  
Parent Finished  
bindu@Bindusree:~$
```



```
GNU nano 8.7.1 zombie.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child Exiting\n");
        return 0;
    }
    else
    {
        sleep(15);
        printf("Parent Finished\n");
    }
    return 0;
}
```

8.orphanprocess


```
bindu@Bindusree: ~  
GNU nano 8.7.1 orphanprocess.c  
#include <stdio.h>  
#include <unistd.h>  
int main()  
{  
    pid_t pid = fork();  
    if (pid > 0)  
    {  
        printf("Parent Exiting\n");  
        return 0;  
    }  
    else  
    {  
        sleep(5);  
        printf("Child PID : %d\n", getpid());  
        printf("New Parent PID : %d\n", getppid());  
    }  
    return 0;  
}  
  
bindu@Bindusree:~$ nano orphanprocess.c  
bindu@Bindusree:~$ gcc orphanprocess.c -o orphanprocess  
bindu@Bindusree:~$ ./orphanprocess  
Parent Exiting  
bindu@Bindusree:~$ Child PID : 1159  
New Parent PID : 337
```